

coretrustbypass_perfectcheater(PerfectCheater)

Sha0huaZhang

我们已经知道，著名的CoreTrust绕过漏洞已经有两个：CVE-2022-26766和CVE-2023-41991。他们分别为TrollStore第1代和第二代所利用的核心CoreTrust漏洞。PerfectCheater（为了便于阅读，下文中均用PerfectCheater代替coretrustbypass_perfectcheater，同时PerfectCheater也是漏洞名称）和CVE-2023-41991很相似，我们不妨先回顾一下CVE-2023-41991，以及Apple的修复存在哪些漏洞。

说到CVE-2023-41991，我们一定会想到“多签名者验证逻辑缺陷”，我们不妨用表格演示这个特殊的签名。

\	SHA-1 CodeDirectory	SHA-256 CodeDirectory
Hash Value	A System App's	Attacker's
Signer	Apple Inc.	Attacker
Binary	The System App	Attacker's App

它巧妙的利用了CoreTrust在处理签名时“拼接”SHA-1的权限（系统App具有或被CoreTrust授予的）和SHA-256的执行内容（攻击者的SHA-256指向的二进制）的错误，让CoreTrust将一个普通的（甚至是恶意的）App误认为系统App，同时CoreTrust还会授予Entitlements.plist的所有权限（除非在当前iOS版本被禁用）。

CVE-2023-41991的影响版本为iOS14.0 beta2-16.7 RC(20H18)和17.0。

Apple在iOS16.7(20H19)和17.0.1的修复策略是：额外增加检查，验证SHA-1和SHA-256的签名者是否一致，从而在根本上阻断了上述漏洞的利用（可是真的是这样吗）。

但是，我们发现：CoreTrust判断一个App是否为系统App时，验证的是哈希值（SHA-1），这个过程与签名者完全无关，甚至使用自己的（或任意有效的）证书签名一个系统App都可以通过CoreTrust的验证。因为篡改后的哈希值无法和系统App保持一致（如果不考虑碰撞），所以大家认为它还是比较安全的。可是它和CoreTrust的“拼接”组合，却催生了下面的新漏洞（PerfectCheater）：

攻击者构造如下签名（可以使用ChOma、MachOX等C语言库构造，具体方法暂不演示）：

\	SHA-1 CodeDirectory	SHA-256 CodeDirectory
Hash Value	A System App's	Attacker's
Signer	Attacker	Attacker
Binary	The System App	Attacker's App

具体的漏洞点为：Apple的修复并不完整，虽然验证了签名者是否一致，但并没有验证其身份是否为Apple Inc.正如之前所说的，任意有效证书签名的系统App也可以通过验证，于是“拼接”的漏洞再次显现（可以尝试推理一下，上述的签名确实会通过CoreTrust的验证）。

如果是这样，这个漏洞在最新版本甚至也存在。可惜的是，iOS17.0.2引入了信任缓存验证。AMFI一旦发现签名的哈希值（SHA-1和SHA-256的任何一项）不符时，为了安全性，偏于更保守的结果，因此会直接拒绝这个签名，漏洞失效。

PerfectCheater的影响版本为iOS14.0 beta2–16.7.15（截止2025-07-13，下同）和17.0 beta1–17.0.1，相比于CVE-2023-41991，新增的版本为16.7(20H19)–16.7.15和17.0.1。

由于时间仓促，未免有错误，请不吝指正！

联系方式：

电子邮箱：trollresignertest1@outlook.com

Twitter：@Sha0huaZhang

Mastodon：@Sha0huaZhang@mastodon.social

Sha0huaZhang
2026-07-13